Varshini Girugula

2403A51L14

Batch-51

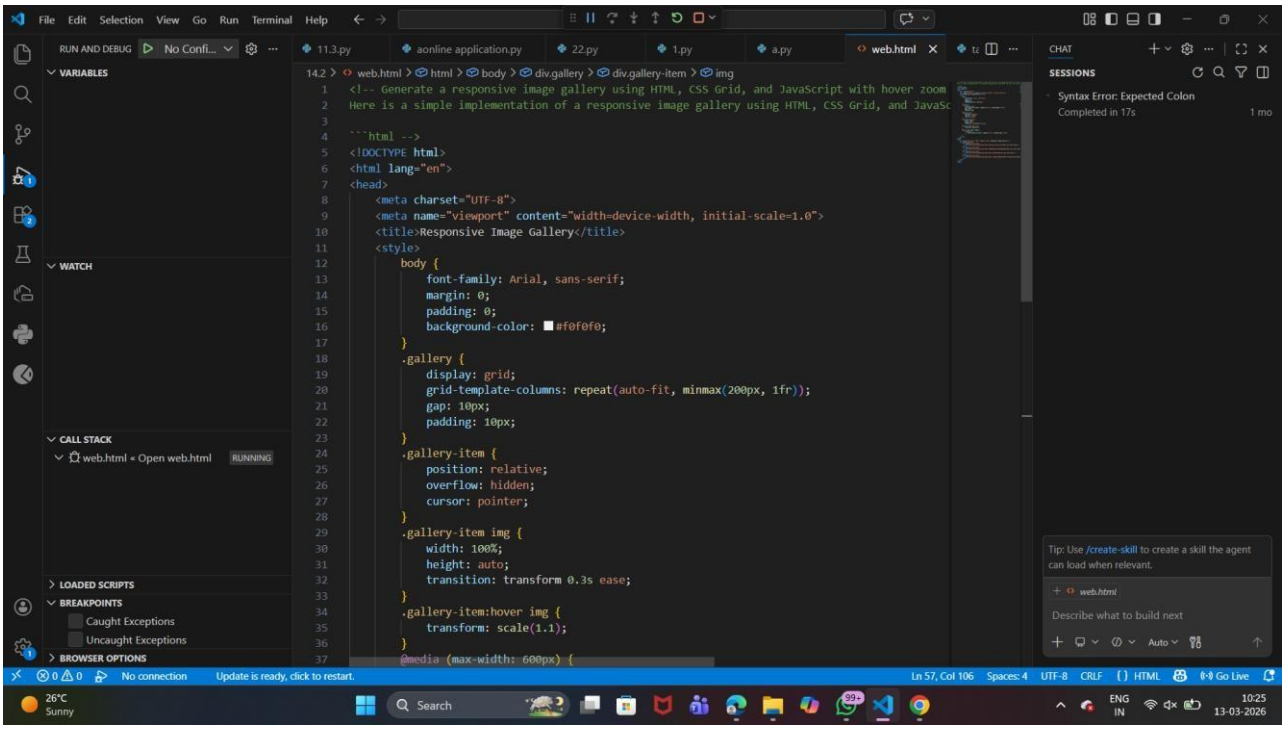
ASSIGNMENT 14.5

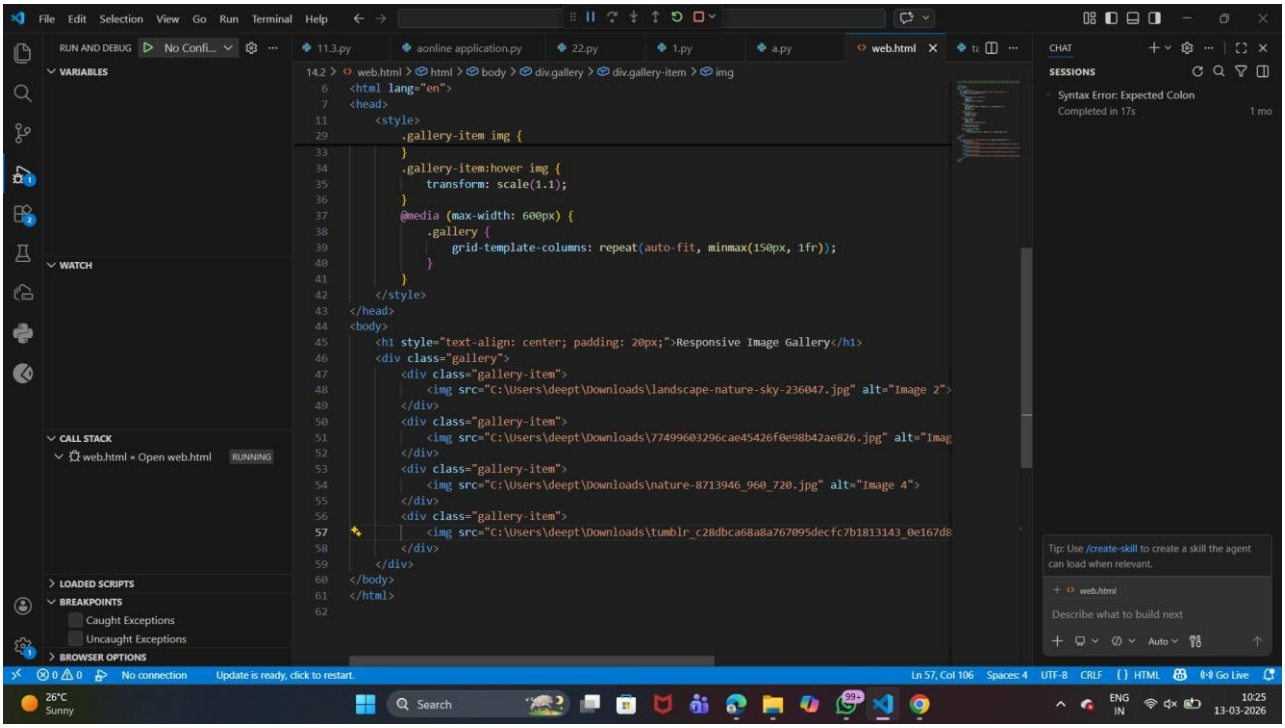
Web Design Application – AI-Assisted HTML/CSS/JS Generation

Task Description #1 (Image Gallery Website):

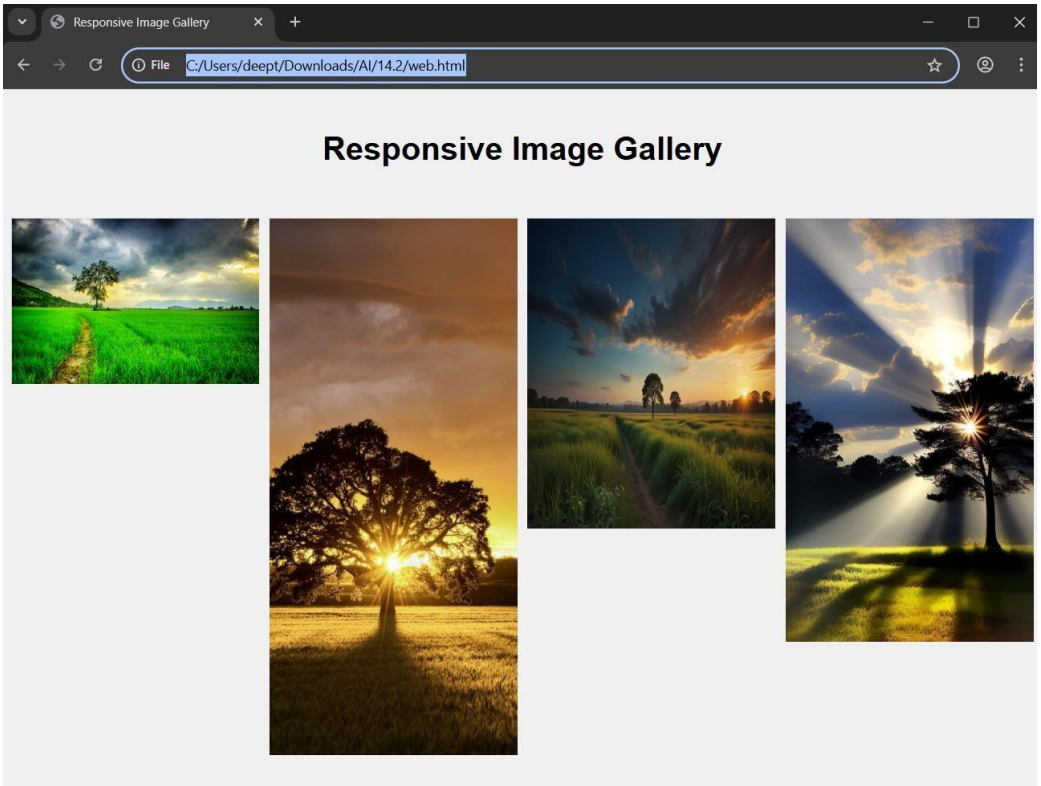
PROMPT: Generate a responsive image gallery using HTML, CSS Grid, and JavaScript with hover zoom effects and mobile-friendly layout.

CODE:





OUTPUT:



Explanation

- CSS Grid arranges images in a structured grid.
- Hover zoom effect improves visual interaction.
- Responsive design adjusts layout for different devices.

Task Description #2 (Profile Card Generator):

PROMPT: Create a modern profile card using HTML, CSS Flexbox, and JavaScript showing user image, name, role, and social media links with hover animation.

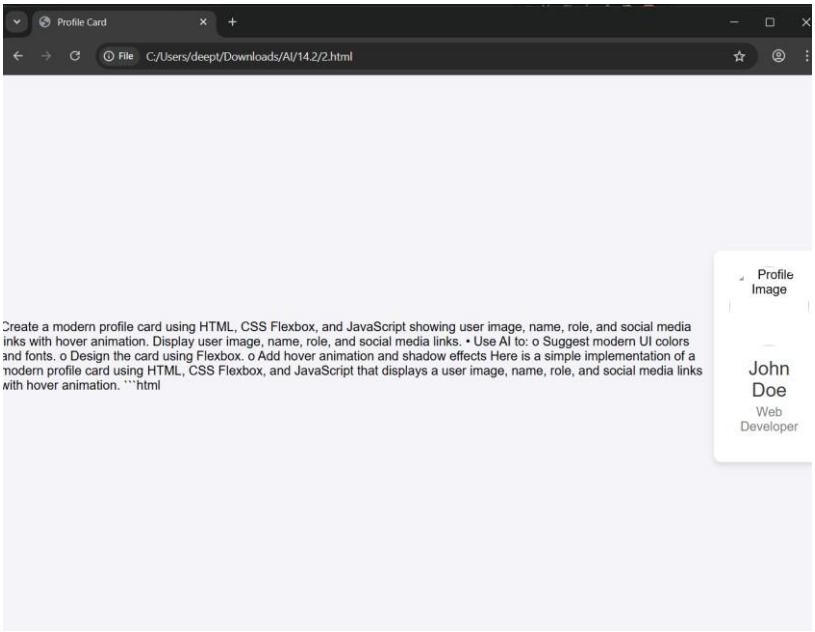
CODE:

```html
<!-- Create a modern profile card using HTML, CSS Flexbox, and JavaScript showing user image,
Display user image, name, role, and social media links.
Suggest modern UI colors and fonts.
Design the card using Flexbox.
Add hover animation and shadow effects
Here is a simple implementation of a modern profile card using HTML, CSS Flexbox, and JavaScri
```html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Profile Card</title>
    <style>
        body {
            font-family: 'Arial', sans-serif;
            background-color: #f4f4f9;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }
        .profile-card {
            background-color: #fff;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            width: 300px;
            text-align: center;
            padding: 20px;
            transition: transform 0.3s ease, box-shadow 0.3s ease;
        }
        .profile-card:hover {
            transform: translateY(-10px);
            box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
        }
        .profile-image {
```

```html
<html lang="en">
<head>
    <style>
        }
        .profile-role {
            font-size: 1em;
            color: #777;
            margin-bottom: 15px;
        }
        .social-links a {
            text-decoration: none;
            color: #555;
            margin: 0 10px;
            font-size: 1.2em;
        }
        .social-links a:hover {
            color: #0077b5; /* LinkedIn Blue */
        }
    </style>
</head>
<body>
    <div class="profile-card">
        <img src="https://via.placeholder.com/100" alt="Profile Image" class="profile-image">
        <div class="profile-name">John Doe</div>
        <div class="profile-role">Web Developer</div>
        <div class="social-links">
            <a href="#" title="LinkedIn"><i class="fab fa-linkedin"></i></a>
            <a href="#" title="Twitter"><i class="fab fa-twitter"></i></a>
            <a href="#" title="GitHub"><i class="fab fa-github"></i></a>
        </div>
    </div>

    <!-- Font Awesome for social media icons -->
    <script src="https://kit.fontawesome.com/a076d05399.js" crossorigin="anonymous"></script>
</body>
</html>
```

OUTPUT:
```



Explanation

- Flexbox aligns card elements properly.
- Modern colors and fonts improve UI design.
- Hover animation adds interactive effects.

Task Description #3 (Develop a dynamic shopping cart system):

Prompt : Develop a dynamic shopping cart using HTML, CSS, and JavaScript that allows adding, removing, and updating items with real-time total price and localStorage support.

Code :

pplication.py | 22.py | 1.py | a.py | web.html | 2.html | 3.html

14.2 > 3.html > html > head > style > body

```html
<!-- Develop a dynamic shopping cart using HTML, CSS, and JavaScript that allows adding, remov
Manage cart state using JavaScript objects.
o Persist cart data in localStorage.
o Update total price in real-time as items are added, removed, or updated.
Here is a simple implementation of a dynamic shopping cart using HTML, CSS, and JavaScript tha
```html --> -->
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Shopping Cart</title>

<style>
body{
    font-family: Arial;
    background: #f4f4f4;
    margin:20px;
}

.container{
    max-width:800px;
    margin:auto;
}

.product, .cart-item{
    display:flex;
    justify-content:space-between;
    background: white;
    padding:10px;
    margin:10px 0;
    border-radius:6px;
}

button{
    padding:5px 10px;
    cursor:pointer;
```

pplication.py | 22.py | 1.py | a.py | web.html | 2.html | 3.html

14.2 > 3.html > html > head > style > body

```html
<html lang="en">
<head>
<style>
}
</style>
</head>

<body>

<div class="container">

<h2>Products</h2>

<div class="product">
<span>Phone - ₹20000</span>
<button onclick="addToCart('Phone',20000)">Add</button>
</div>

<div class="product">
<span>Laptop - ₹50000</span>
<button onclick="addToCart('Laptop',50000)">Add</button>
</div>

<div class="product">
<span>Headphones - ₹2000</span>
<button onclick="addToCart('Headphones',2000)">Add</button>
</div>

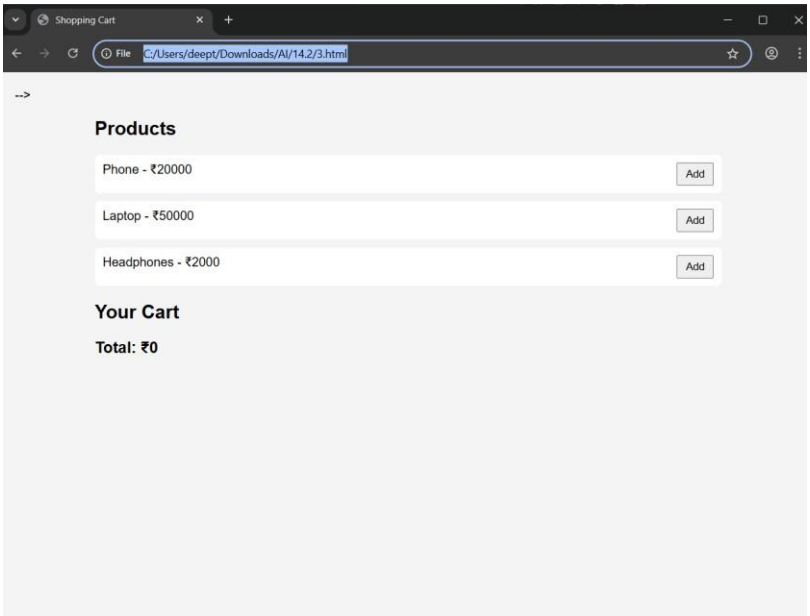
<h2>Your Cart</h2>
<div id="cart"></div>

<div class="total">
Total: ₹<span id="total">0</span>
</div>

</div>
```
```

```html
<html lang="en">
<body>
<script>
function updateCart(){
let total = 0;

for(let item in cart){

let price = cart[item].price;
let qty = cart[item].qty;

total += price * qty;

cartDiv.innerHTML += `
<div class="cart-item">
<span>${item} - ₹${price}</span>

<input type="number" value="${qty}" min="1"
onchange="changeQty('${item}',this.value)">

<button onclick="removeItem('${item}')">Remove</button>
</div>
`;
}

document.getElementById("total").innerText = total;

localStorage.setItem("cart", JSON.stringify(cart));
}

updateCart();

</script>

</body>
</html>
```

Output :



Explanation

- JavaScript objects manage cart items.
- Total price updates automatically.
- localStorage saves cart data after refresh.

Task description #4 (Role-Based Access Control Application):

Prompt : Create a role-based web application using HTML, CSS, and JavaScript with Admin, Editor, and User roles and conditional UI rendering and Design access control logic,Maintain secure state transitions,Create scalable UI components.

Code :

```html
<!-- Create a Role-Based Access Control web app using HTML, CSS, and JavaScript.
Roles: Admin, Editor, User.
Show different UI sections based on role using conditional rendering.
Here is a simple implementation of a Role-Based Access Control (RBAC) web app using HTML, CSS,
```html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Role-Based Access Control</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f4;
            margin: 0;
            padding: 20px;
        }
        .container {
            max-width: 600px;
            margin: auto;
            background-color: #fff;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
        }
        .section {
            display: none;
        }
        .admin-section {
            background-color: #e3f2fd;
            padding: 10px;
            margin-bottom: 10px;
        }
        .editor-section {
            background-color: #fff3e0;
            padding: 10px;
```

```html
<html lang="en">
<body>
    <div class="container">
        <select id="role" onchange="renderSections()">
            <option value="">--Select Role--</option>
            <option value="admin">Admin</option>
            <option value="editor">Editor</option>
            <option value="user">User</option>
        </select>

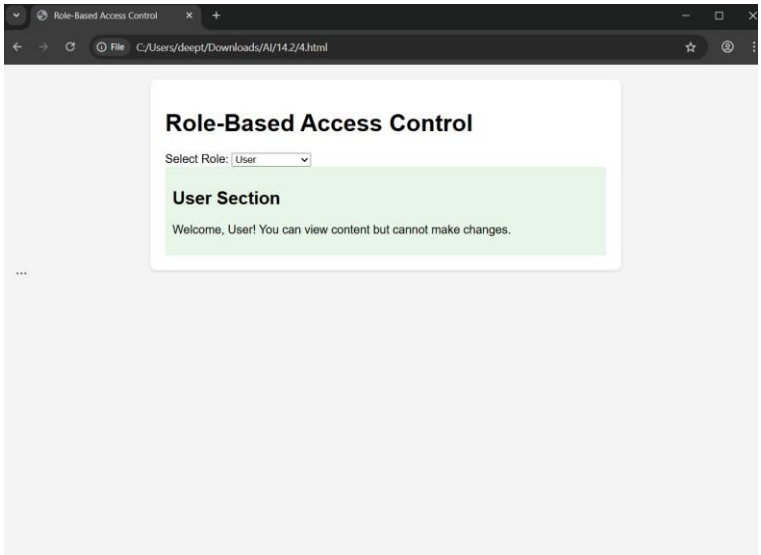
        <div class="section admin-section" id="adminSection">
            <h2>Admin Section</h2>
            <p>Welcome, Admin! You have full access to the system.</p>
        </div>

        <div class="section editor-section" id="editorSection">
            <h2>Editor Section</h2>
            <p>Welcome, Editor! You can edit content but have limited access to settings.</p>
        </div>

        <div class="section user-section" id="userSection">
            <h2>User Section</h2>
            <p>Welcome, User! You can view content but cannot make changes.</p>
        </div>
    </div>

    <script>
        function renderSections() {
            const role = document.getElementById('role').value;
            document.getElementById('adminSection').style.display = role === 'admin' ? 'block'
            document.getElementById('editorSection').style.display = role === 'editor' ? 'bloc
            document.getElementById('userSection').style.display = role === 'user' ? 'block' :
        }
    </script>
</body>
</html>
```
```

Output :



Explanation

- Different roles have different permissions.
- UI changes based on user role.
- Improves security and access control.

Task Description #5 (Multi-Language Website-Design a multilingual website):

Prompt : Design a multilingual website using HTML, CSS, and JavaScript with dynamic language switching and language preference stored in localStorage and Structure translation files,Handle RTL/LTR layouts,Improve accessibility.

Code :

1.py a.py web.html 2.html 3.html 4.html 5.html

14.2 > 5.html > html > head > style > .container

```html
<!-- Develop a multilingual website using HTML, CSS, and JavaScript.
Allow dynamic language switching.
Store translations in JSON and save language preference in localStorage.
Here is a simple implementation of a multilingual website using HTML, CSS, and JavaScript that
```html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Multilingual Website</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f4;
            margin: 0;
            padding: 20px;
        }
        .container {
            max-width: 600px;
            margin: auto;
            background-color: #fff;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
        }
        .language-selector {
            margin-bottom: 20px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1 id="title">Welcome to our website!</h1>
        <p id="description">This is a simple multilingual website example.</p>
        <div class="language-selector">
            <label for="language">Select Language:</label>
```

1.py a.py web.html 2.html 3.html 4.html 5.html

14.2 > 5.html > html > head > style > .container

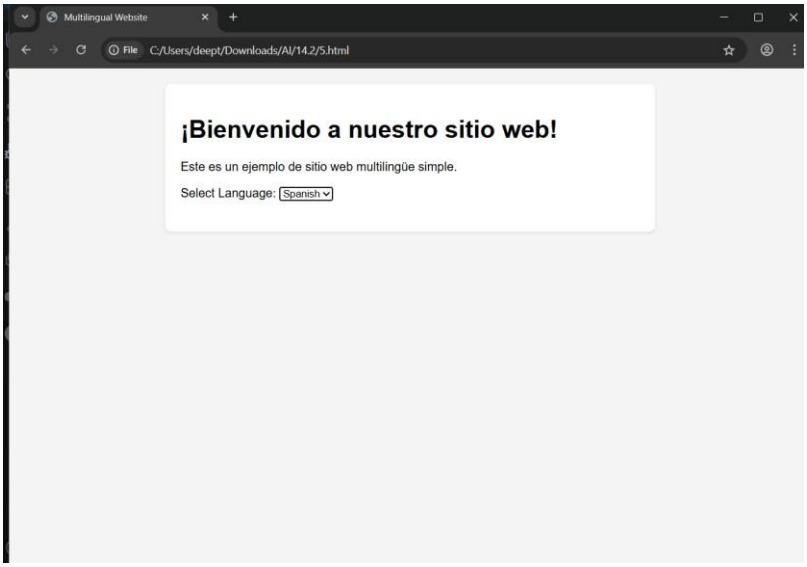
```html
<html lang="en">
<body>
    <div class="container">
    </div>

    <script>
        const translations = {
            en: {
                title: "Welcome to our website!",
                description: "This is a simple multilingual website example."
            },
            es: {
                title: "¡Bienvenido a nuestro sitio web!",
                description: "Este es un ejemplo de sitio web multilingüe simple."
            },
            fr: {
                title: "Bienvenue sur notre site Web!",
                description: "Ceci est un exemple de site Web multilingue simple."
            }
        };

        function switchLanguage() {
            const language = document.getElementById('language').value;
            document.getElementById('title').textContent = translations[language].title;
            document.getElementById('description').textContent = translations[language].descri
            localStorage.setItem('preferredLanguage', language);
        }

        // Load preferred language on page load
        window.onload = function() {
            const preferredLanguage = localStorage.getItem('preferredLanguage') || 'en';
            document.getElementById('language').value = preferredLanguage;
            switchLanguage();
        };
    </script>
</body>
</html>
```

Output :
```



Explanation:

- JSON files store translation data.
- Users can switch languages dynamically.
- Language preference is saved for future visits.

Task Description #6 (Online Examination System-Create an online exam interface.):

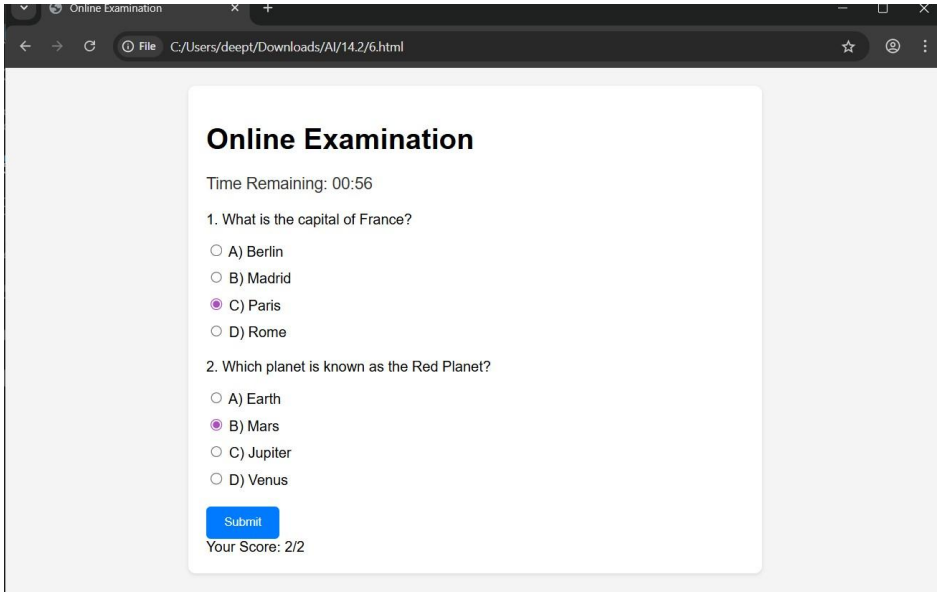
Prompt : Create an online examination interface using HTML, CSS, and JavaScript with MCQ questions, countdown timer, and single submission system and Manage exam state,Improve exam UX,Ensure accessibility.

Code :

```html
<html lang="en">
<head>
    </style>
</head>
<body>
    <div class="container">
        <h1>Online Examination</h1>
        <div class="timer" id="timer">Time Remaining: 01:00</div>
        <div class="question">
            <p>1. What is the capital of France?</p>
            <ul class="options">
                <li><input type="radio" name="q1" value="A"> A) Berlin</li>
                <li><input type="radio" name="q1" value="B"> B) Madrid</li>
                <li><input type="radio" name="q1" value="C"> C) Paris</li>
                <li><input type="radio" name="q1" value="D"> D) Rome</li>
            </ul>
        </div>
        <div class="question">
            <p>2. Which planet is known as the Red Planet?</p>
            <ul class="options">
                <li><input type="radio" name="q2" value="A"> A) Earth</li>
                <li><input type="radio" name="q2" value="B"> B) Mars</li>
                <li><input type="radio" name="q2" value="C"> C) Jupiter</li>
                <li><input type="radio" name="q2" value="D"> D) Venus</li>
            </ul>
        </div>
        <button class="submit-btn" id="submitBtn">Submit</button>
        <div id="result"></div>
    </div>
    <script>
        let timeLeft = 60;
        const timerElement = document.getElementById('timer');
        const submitBtn = document.getElementById('submitBtn');
        const resultElement = document.getElementById('result');
        let timerInterval;

        function startTimer() {
```

```html
<html lang="en">
<body>
    <script>
        function startTimer() {
            timerInterval = setInterval(() => {
                submitExam();
            } else {
                timeLeft--;
                const minutes = Math.floor(timeLeft / 60).toString().padStart(2, '0');
                const seconds = (timeLeft % 60).toString().padStart(2, '0');
                timerElement.textContent = `Time Remaining: ${minutes}:${seconds}`;
            }
        }, 1000);
    }

    function submitExam() {
        submitBtn.disabled = true;
        const q1Answer = document.querySelector('input[name="q1"]:checked')?.value;
        const q2Answer = document.querySelector('input[name="q2"]:checked')?.value;
        let score = 0;

        if (q1Answer === 'C') score++;
        if (q2Answer === 'B') score++;

        resultElement.textContent = `Your Score: ${score}/2`;
    }

    submitBtn.addEventListener('click', () => {
        clearInterval(timerInterval);
        submitExam();
    });

    startTimer();
    </script>
</body>
</html>
```

Output :



Explanation :

- Timer controls exam duration.
- JavaScript manages exam state.
- Prevents multiple submissions.